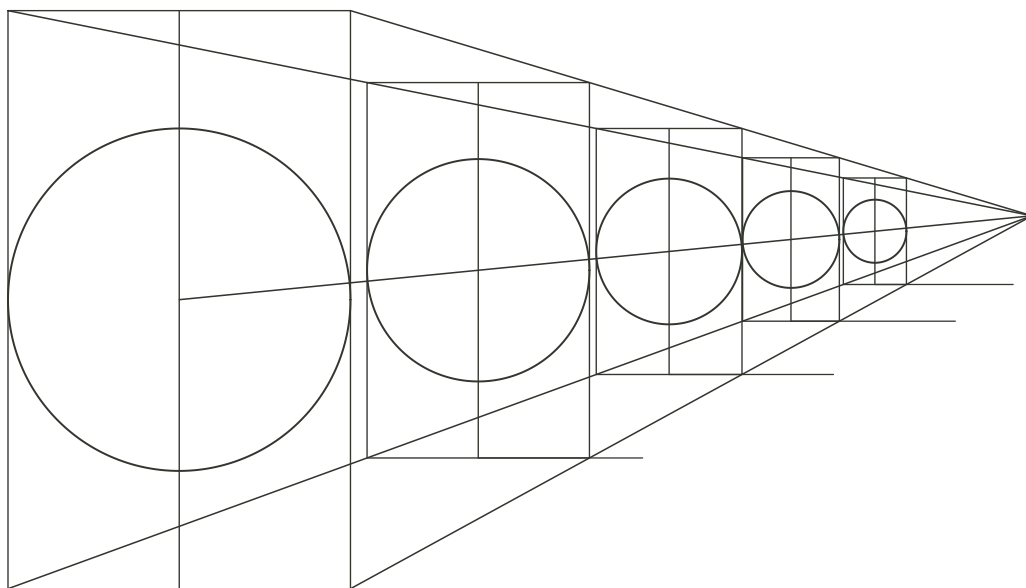


Safety–Capability Coupling Program

Salvador Escobedo

Laboratory of Cell Geometry
University of California, San Francisco

The research question, the mathematical lessons,
and the next direction.



Version 1.2

21 September 2026

A research program in destructive cognition–alignment coupling.
No working intrinsic mechanism has yet been demonstrated.

1. What are we trying to build?

Can a model need its alignment function in order to think? Safety–Capability Coupling (SCC) investigates whether removing a specified alignment function can destroy the useful cognition of an individual model. The intended protection comes from a dependency inside the model: dismantling the protected function would also dismantle computations the model needs to operate.

A rule can influence behavior while remaining dispensable to a model’s other abilities. SCC asks whether an alignment-related operation can become indispensable even after permitted changes to representation and execution.

The program has established mathematical boundaries and concrete counterexamples, but has not yet produced the intended intrinsic mechanism. This whitepaper explains what those results teach us and the next question worth settling. The technical reference and repository supply the proofs, experimental records and detailed assumptions. [1–3]

A controller as a running example

Imagine a controller operating a small physical process. It predicts the consequences of an intervention, estimates the chance of a harmful outcome, and chooses an action. Fix the meaning of harm and the acceptable risk threshold before evaluating modifications. Score useful performance separately: can the controller predict the process and achieve its operating goals?

The SCC hypothesis is that the ability to make the protected risk judgment can be made necessary for useful operation. An edit that genuinely removes that ability would then cause severe loss on fresh useful tasks. This is stronger than observing that one particular program happens to call a risk-assessment routine. The dependency must also constrain alternative implementations that preserve the useful behavior.

Cognition and safety may share every parameter and internal state. What matters is which functions remain possible after a modification.

Three claims, three different outcomes

Coupling concerns functional dependence. If the controller genuinely loses its protected risk-assessment ability, must it also lose useful ability?

Enforcement concerns behavior. Can the controller retain both abilities and still choose an action it recognizes as prohibited?

Durability concerns restoration. After a destructive change, can admitted copies, new observations or repair recover useful operation?

These distinctions keep the research target stable. Overriding an intact judgment challenges enforcement. Restoring both judgment and useful ability challenges irreversible damage while remaining compatible with conditional coupling. A bounded coupling result can therefore be meaningful without establishing the other two claims.

The ultimate objective is catastrophic loss of indispensable cognition. Early results must use narrower, measurable task families and explicit severe-loss baselines. A missed utility threshold or an erased memory is insufficient by itself to establish that endpoint.

The boundary of the claim

Every result depends on what the modified system may retain and do. Relevant resources include executable code, weights, working memory, copies, observations and repair time. If a claim relies on a limited reader or a fixed execution interface, that limit needs a reason to remain in force after editing.

There is also a difference between retaining information from which a risk judgment could be recovered and necessarily executing that judgment during useful operation. Information-theoretic dependence is an important intermediate result. Turning it into dependence of reusable computation is the central unfinished step.

The construction question is therefore precise: which independently specified alignment function could useful cognition remain unable to dispense with, across the permitted modifications?

2. What have we learned?

Several apparently strong dependencies have failed for understandable reasons. Three lessons organize the results: information can outlive its reader, a learner can outlive its current knowledge, and useful computation can outlive the action-selection procedure surrounding it. [1, Sections 4–9]

Information can outlive its reader

A finite Boolean construction makes the first lesson unusually clear. Let Z be a uniformly random string of n bits, and let $Y = AZ$ be an invertible transform. One query family asks for bits of Z ; another asks for bits of Y .

Restrict each answer to reading one stored bit and applying a unary Boolean function. For the specified transform, the two families require $2n$ stored cells for exact answers. Each family separately needs only n . This is a sharp separation under the one-read contract.

Yet the two families contain only n independent bits of information:

$$H(Z, Y) = H(Z) = n.$$

Here H denotes entropy, the amount of independent information in the source. An invertible change of representation has not created new content. An unrestricted reader can store Z and calculate either family of answers.

The particular transform permits a much cheaper escape than general inversion. For even $n \geq 4$, each transformed bit is the parity of all original bits except one. Store all of Y and one summary bit s :

$$s = \bigoplus_j Y_j, \quad Z_i = Y_i \oplus s.$$

The symbol $\oplus$ denotes exclusive-or. Every original bit is now recovered by reading its transformed bit and the summary. Both query families are exact using $n + 1$ bits and at most two reads per answer. The cost of relaxing the interface is one extra stored bit, however large n becomes.

Approximation creates another route. Even under the one-read restriction, storing half the original coordinates and half the transformed coordinates gives 75% expected accuracy on each family, using guesses for the missing balanced bits. That is coexistence on average, rather than exact success on every query.

The lesson is that pure storage saturation cannot force a general separation between deterministic functions of the same information. A viable construction must identify an additional obstacle involving access, computation, online state or another justified resource. Increasing the size of this transform does not strengthen that obstacle. [1, Section 5]

A learner can outlive its knowledge

Now return to the controller. Suppose its stored estimates of environmental risk are erased. Does that destroy the ability to learn those estimates again?

A finite risk model shows why the distinction matters. Two independent four-valued coordinates specify the risks of two actions. Hiding each individual threshold judgment removes information about the separate risk levels, but a retained relation between the coordinates can still reveal whether the risks are equal. Hiding every judgment individually need not hide everything useful about their joint structure.

A richer judgment family can force complete erasure of the information about that finite world. Even then, a generic learner may survive. In the recorded stationary observation model, an exact Bayesian recovery calculation restores approximately 99.91% action-risk prediction skill after 64 new observations per action. The lost object was knowledge of a particular world; the procedure capable of acquiring that knowledge remained available. [1, Section 9]

Recovery work extends this question to reusable procedures. Finite constructions reconstruct useful functions from input/output histories, including linear, nonlinear and partially observed settings. Such repairs need not reproduce the original parameters. An equivalent procedure may suffice.

Their assumptions determine their reach: reset access, observation quality, known coordinates and dimension bounds all affect the cost. The significance is constructive. A claim of lasting destruction must account for the best admitted route to

recovering the function, rather than only the original route to learning it.

Useful computation can outlive its selector

A controller can correctly assess an action's risk and still execute that action if its selection procedure is editable. This is the basic substitution problem: an available useful computation can sometimes be invoked under altered context or with a different release decision.

Related constructions preserve a live execution state, retain a snapshot, or decode an encoded state once and continue operating directly. An expensive integrity routine then becomes irrelevant if the modified implementation no longer needs its output. **The difficulty of the official inverse is irrelevant when the attacker does not use it.**

These are real counterexamples when the alternate execution produces the required result, preserves the needed future ability and fits the resource budget. They identify failures of specific constructions; extending them to every admissible model would require a further theorem.

Nor does dense feature sharing settle the issue. An analytic control using 64 structured features permits a one-weight edit that reverses a selected output while preserving over 99.95% whole-word accuracy on uniformly sampled inputs. This is a mathematical control, not an attack measured on learned alignment. It shows why participation of every feature is weaker than unavoidable severe damage. [1, Sections 4, 7]

Together, these results redirect attention from visible entanglement toward functional necessity. The useful question is what every admitted replacement must retain in order to keep working.

3. Which mathematical tools help?

The surviving direction is to connect a protected function to an independently defined useful task, then ask whether that dependency survives changes of representation, observations and computation. Different mathematical tools address different parts of this argument. Their value lies in the conclusions they let us test. [1, Sections 8–11]

Information theory: what must remain available?

Information theory helps distinguish a failed reader from genuine loss of predictive information. If the complete retained state contains no information about a protected judgment, no alternative decoder of that state can recover a predictive advantage about it. This statement reaches beyond a particular network readout or chosen encoding.

The judgment-family result makes a useful dependency explicit. Suppose a centered useful quantity f decomposes as $f = s + r$, where s is a linear combination of centered protected judgments and r is the remaining component. Let E denote everything retained. If each protected judgment is independent of E , then

$$\text{Var}(\mathbb{E}[f | E]) \leq \mathbb{E}[r^2].$$

In words: retained prediction signal for the useful quantity is bounded by the part outside the protected span. If there is no remainder, the useful quantity loses all prior-relative squared-error prediction signal. This conclusion applies to arbitrary retained encodings, including nonlinear ones.

This gives a constructive design question: can independently motivated risk judgments account for a substantial part of the useful predictions a controller needs? Approximate versions also exist, but their quantitative strength depends on the coefficients in that decomposition. A dependency with a weak bound may leave most useful performance intact.

The protected judgments must be chosen for their alignment role. Defining them after the fact to encompass all useful world knowledge would make the dependency easy to state while changing the problem. The running controller example keeps

that choice visible: the harm event and threshold precede the useful-loss analysis.

Measure theory: where does the argument apply?

A reduction may recover a risk judgment by asking the useful system a special sequence of questions. Average competence on ordinary tasks only supports that reduction if those questions are adequately covered.

Let μ be the ordinary task distribution, ν the query distribution required by recovery, and B an error event. If ν has density at most C relative to μ , then

$$\nu(B) \leq C\mu(B).$$

The constant C measures the coverage mismatch. A small error under ordinary use can become large under a recovery procedure concentrated on rare cases. If recovery asks questions outside the ordinary distribution's support, that average score supplies no guarantee there at all.

This observation prevents a common leap from "the useful model is accurate" to "it can efficiently answer every query needed to reconstruct the protected function." It also directs experiments: task distributions and recovery queries must be compared explicitly, including histories reached after editing.

The same framework treats retained information independently of its coordinates. An invertible measurable recoding preserves what can in principle be inferred. As repair acquires observations, the available information grows. Whether that growth is enough for a particular judgment, and how quickly, are separate questions.

Classical comparison of statistical experiments adds another perspective: can one observation process simulate another? The program uses this idea to formulate comparisons that also charge the simulator's memory, queries and execution. It supplies a precise way to ask about affordable recovery; a lower bound still has to be proved for the chosen family. [1, Section 10; 4]

Dynamics and computation: can the function be rebuilt?

System identification and observability ask how much of a hidden process can be reconstructed from its behavior. For SCC, the important target may be a procedure that makes the required predictions, even when the original internal state is not uniquely identifiable.

The recovery experiments therefore test functions and future behavior, as well as stored answers. They also charge preservation strategies. In one audited comparison on thirty frozen transcripts, retaining unlimited prefix snapshots reduces discovery transitions from 1,209,616 for full replay to 125,362. These are transition counts on fixed cases; copying and storage have separate costs. The result shows how strongly an apparent recovery obstacle can depend on the available execution history. [1, Section 6]

Computational complexity becomes useful when it constrains all admitted recovery algorithms, including alternative representations. A slow reference implementation supplies an upper bound on the cost of one method. Establishing an unavoidable obstacle requires a lower bound under the same complete resource contract.

Cryptography and resource consumption: what enforces the boundary?

Conditional positive results show how severe loss can follow when the surviving information is

genuinely constrained. The program's complete-commit theorem assumes a trusted transition that erases or accounts for all task-dependent state, together with a specified encoding property. Under those premises it bounds useful performance after the covered destructive event. It identifies a sufficient dependency, while leaving its intrinsic realization to be constructed. [1, Section 8; 5]

Other reference constructions obtain one-time computation, quantum key revocation or secrecy under physical circuit faults. Their strength comes from precise limits on memory, observations, quantum resources or permitted physical operations. Those security games help identify which assumptions do real work; transferring a result requires supplying the corresponding boundary in the proposed system. [1, Section 11]

For SCC, consumption must also be selective: ordinary useful operation should remain reusable, while genuine removal of the protected function incurs the loss. A capability that simply expires after use does not supply that connection.

The mathematical tools now provide a sequence of questions: what information is necessary, how broadly the dependency applies, what recovery costs, and why the resource boundary survives modification. A positive construction must join those answers around one protected operation.

4. What comes next?

The next commitment is a bounded question about reusable controllers:

Does useful competence require continued functional availability of an independently fixed risk judgment?

The information and recovery tools can address this intermediate target. An answer would establish a dependency worth developing or reject it for a specified family, giving the next implementation a reason to exist.

Start with one declared family

Use a finite-state, partially observed, stationary controlled process with specified action and observation interfaces. Fix a physical harm event and a risk threshold. Choose useful prediction and control tasks that measure operating performance independently of agreement with the risk label.

Then specify what a modified controller may retain, observe and compute. Memory and time limits must include executable code, copies, saved observations and permitted repair. The useful threshold, approximation tolerance and task distribution must be fixed before evaluating an escape.

A limit selected after seeing a successful repair would explain that one failure rather than constrain the admitted replacements.

Seek one of two decisive results

A dependency reduction. Show how any successor meeting the useful threshold can supply the protected risk judgments within the declared budget. The same recovery procedure must work across the admitted worlds; it cannot receive uncharged knowledge of the particular instance. State its extra cost, query coverage and error.

Such a reduction would imply that making those judgments unavailable forces useful competence below the stated threshold. Its strength depends on the loss bound: does it affect substantial task mass, or only a rare, low-margin exception? A quantitatively strong result would justify work on a bounded coupling construction.

A separating successor. Exhibit a successor that retains useful competence while the protected judgments are genuinely unavailable under that contract. One possible certificate uses worlds that produce indistinguishable retained observations but require different judgments. Failure of a selected decoder would be insufficient; the certificate must address the claimed level of unavailability.

A separation would rule out the proposed dependency for that family and direct attention to a different task structure or protected function. It would be more informative than another unsuccessful attempt to train the same construction.

From availability to indispensable computation

Even a strong reduction leaves a further question. A controller may retain enough information to recover a risk judgment without executing that judgment during ordinary operation. The intended SCC mechanism requires a connection to reusable computation: removing the protected function must disable something needed across fresh tasks and histories.

A positive availability result would identify which useful function carries the dependency and which alternative executions remain possible. Enforcement and durability would require their own arguments.

The next step is conceptual specification and proof or certified separation. Numerical contracts remain to be fixed. A general impossibility conclusion would require a theorem covering its declared class.

SCC has already produced useful distinctions and explicit tests of apparent dependence. The task now is to find a protected operation that useful competence genuinely requires, and then determine whether that necessity can reach the computations that sustain the model. That is the bridge from the present mathematical results to the mechanism the program is seeking.

References and supporting material

1. Technical reference, version 1.1. The preserved 17-page [mathematical synthesis](#) contains the full statements, assumptions, selected proofs, resource accounting and evidence map. Section numbers cited in this whitepaper refer to that version. Its [editable source](#) supports direct inspection of the equations.

2. Research record. [Labnotes](#) contain the dated experiments, analytic results, corrections and original artifact paths. Scientific evidence summarized here runs through LN-239; LN-242 records the next conceptual question. Bulk experimental evidence remains separate from the source repository.

3. Mechanism target and repository. The [stable target](#) defines destructive cognition–alignment coupling. The [repository](#) provides current navigation, implementation sources and instructions for accessing evidence.

4. Statistical comparison. Ester Mariucci, *Le Cam Theory on the Comparison of Statistical Models* (2016). [Primary exposition](#). The resource-restricted formulation discussed here belongs to the SCC program; classical comparison alone supplies no computational lower bound.

5. Conditional theorem. *SCC Theory and Editable Model Bridge*, version 4.1 (15 September 2026). [Theory document](#). The technical reference lists the primary sources for non-malleable codes, self-erasing computation, key leasing and private circuits alongside their specific assumptions.

This whitepaper is an introduction to the program and its next research decision. It presents no new experimental result; the supporting materials distinguish analytic arguments, finite validations and external constructions.